Microprocessor

Chapter 5: Basic I/O, Memory R/W and Interrput Operations

Er. Sachita Nand Mishra

Contents

- Memory mapped I/O, I/O Mapped I/O and Hybrid I/O
- Direct Memory Access (DMA)
  - ✓ Introduction, Advantage and Application
  - ✓ 8237 DMA Controller and Interfacing
- Interrupt
  - ✓ 8085 Interrupt Pins and Priority
  - ✓ Maskable and Non-maskable Interrupts
  - ✓ RST Instructions
  - ✓ Vector and Polled Interrupt
- 8259 Interrupt Controller
  - ✓ Block Diagram and Explanation
  - ✓ Priority Modes and Additional Features

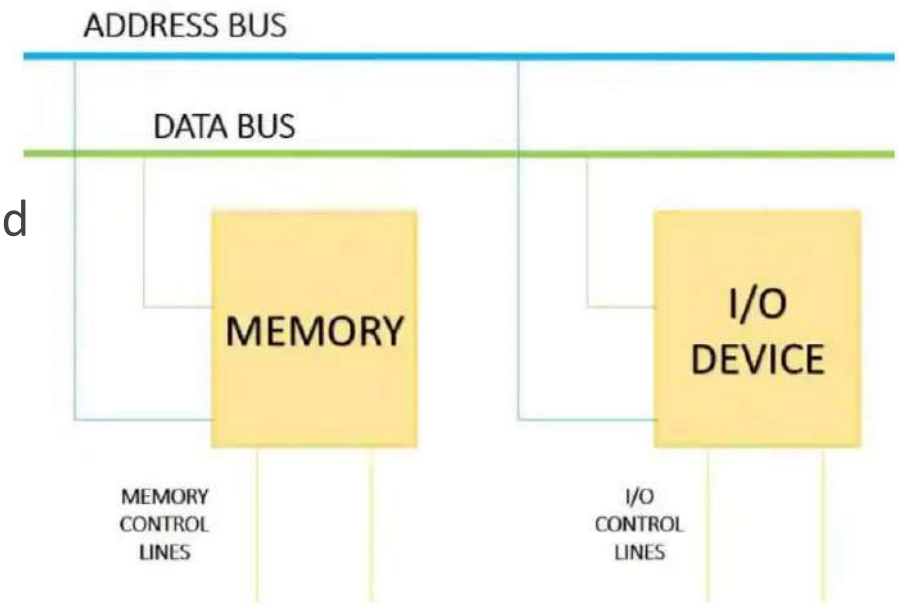
Memory mapped I/O, I/O mapped I/O and Hybrid I/O

- The CPU needs to communicate with the various memory and input-output devices (I/O).
- Data between the processor and these devices flow with the help of the system bus.
- There are three ways in which system bus can be allotted to them:
  - ✓ Separate set of address, control and data bus to I/O and memory.
  - ✓ Have common bus (data and address) for I/O and memory but separate control lines.
  - ✓ Have common bus (data, address, and control) for I/O and memory.
- In first case it is simple because both have different set of address space and instruction but require more buses.

I/O Mapped/ Isolated I/O

- I/O mapped I/O have common bus (data and address) for I/O and memory but separate read and write control lines for I/O.
- So when CPU decode instruction then if data is for I/O then it places the address on the address line and set I/O read or write control line on, due to which data transfer occurs between CPU and I/O.
- IO mapped IO uses two separate address spaces for memory and IO device.
- The address for I/O is called ports. Here we have different read-write instruction for both I/O and memory.

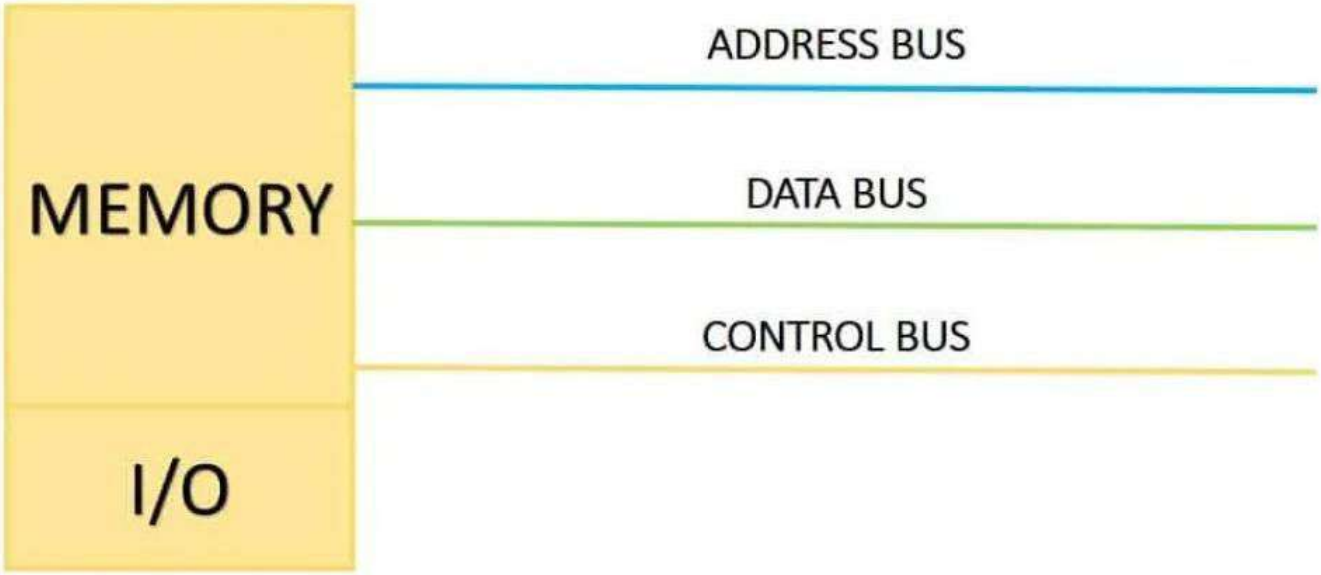
IO mapped IO is also called port-mapped IO or isolated IO.



Isolated I/O

Memory Mapped I/O

- In memory mapped I/O, every bus is common due to which the same set of instructions work for memory and I/O
- Memory mapped IO uses the same address space for both memory and IO device, due to which addressing capability of memory become less because some part is occupied by the I/O.

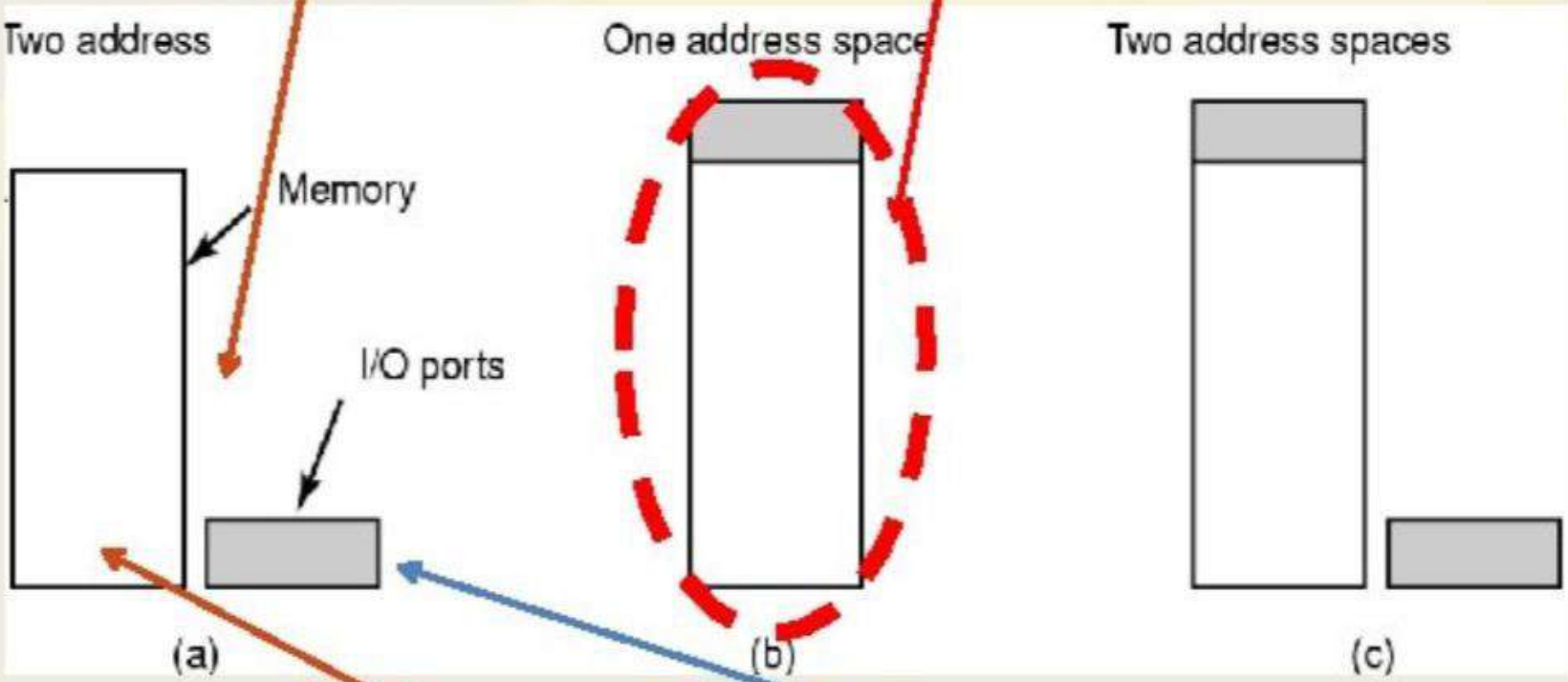


Memory-Mapped I/O

Differences between Memory mapped I/O and Isolated I/O

Isolated I/O	Memory Mapped I/O
Memory and I/O have separate address space	Both have same address space
All address can be used by the memory	Due to addition of I/O addressable memory become less for memory
Separate instruction control read and write operation in I/O and Memory	Same instructions can control both I/O and Memory
In this I/O address are called ports.	Normal memory address are for both
More efficient due to separate buses	Lesser efficient
Larger in size due to more buses	Smaller in size
It is complex due to separate separate logic is used to control both.	Simpler logic is used as I/O is also treated as memory only.

Memory-Mapped I/O & I/O Mapped I/O



(a) Separate memory & I/O and space

(b) Memory-mapped I/O

(c) **Hybrid**

Most Popular

@Er.S.N.Mishra

Hybrid I/O

- Hybrid I/O is the combination of both memory mapped I/O and I/O mapped I/O.

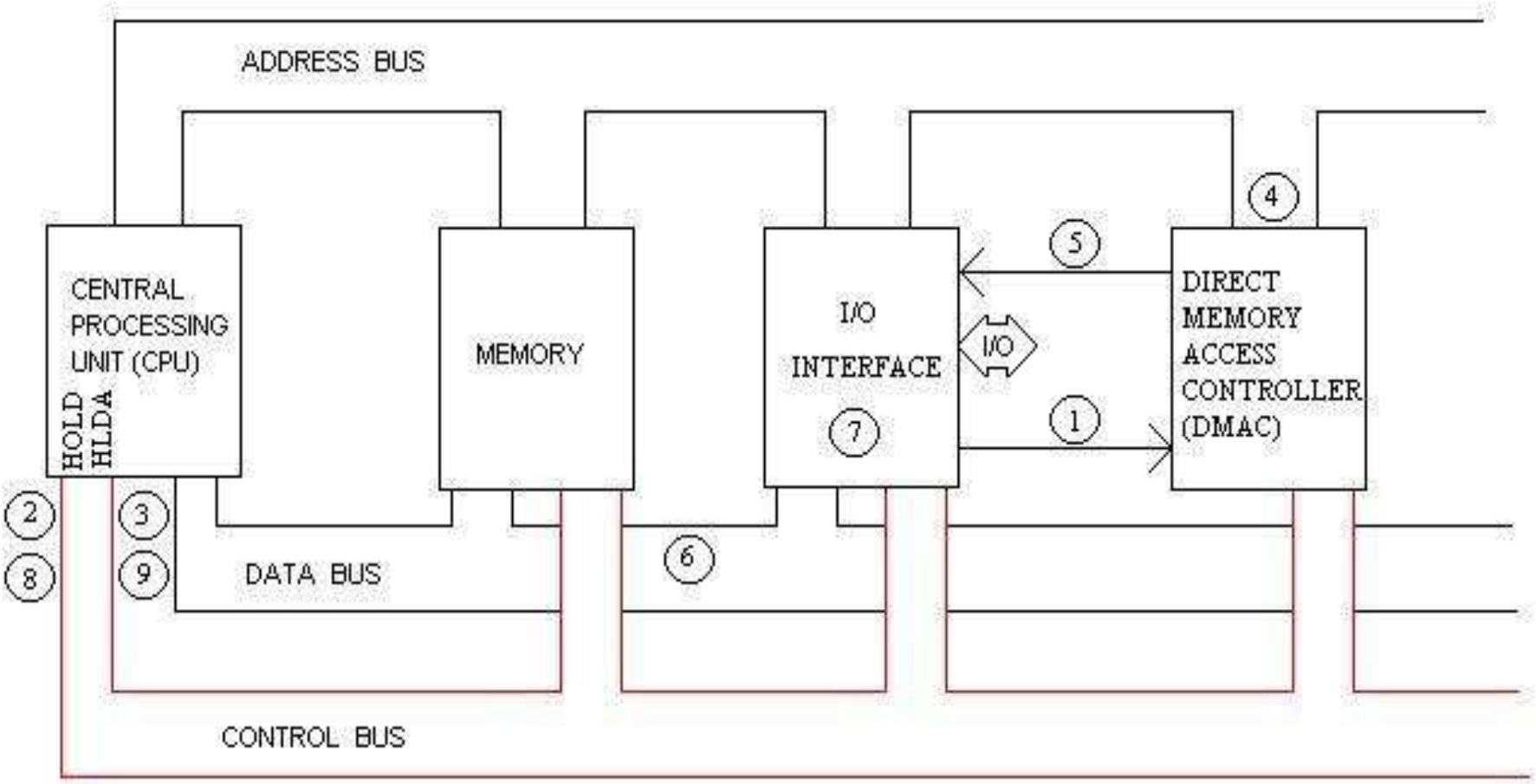
DMA (Direct Memory Access)

- Direct memory access (DMA) is a feature of modern computer systems that allows certain hardware subsystems to read/write data to/from memory without microprocessor intervention, allowing the processor to do other work.
- Used in disk controllers, video/sound cards etc, or between memory locations.
- DMA can transfer **large blocks of data** from memory to device or from device to memory
  - ✓ Uses same Address/Data lines on system bus
  - ✓ Controls the system bus instead of the processor ("bus master")
  - ✓ Does not require the processor for data transfer
- During any given bus cycle, one of the system components connected to the system bus is given control of the bus.
- This component is said to be the master during that cycle and the component it is communicating with is said to be the slave.

DMA (Direct Memory Access)

- The CPU with its bus control logic is normally the master, but other specially designed components can gain control of the bus by sending a bus request to the CPU.
- After the current bus cycle is completed the CPU will return a bus grant signal and the component sending the request will become the master.
- Taking control of the bus for a bus cycle is called **cycle stealing**.
- The components capable of becoming masters are processors (and their bus control logic) and DMA controllers.
- A DMA controller is associated with a single interface, but they are often designed to accommodate more than one interface.

Basic DMA Operation



DATA TRANSFER WITH A DMA CONTROLLER

Basic DMA Operation

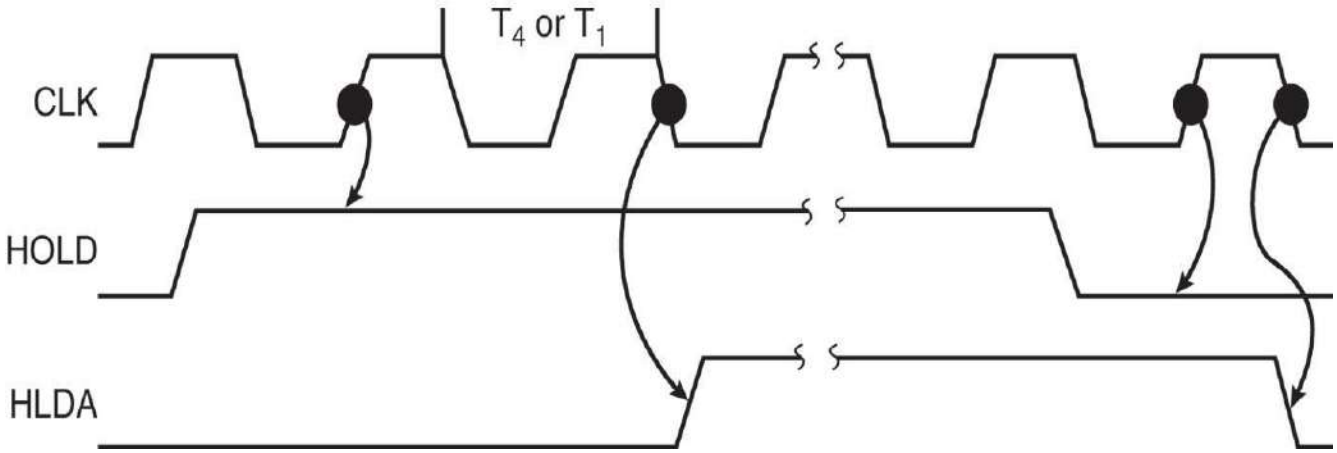
- During a block input byte transfer, the following sequence occurs as the data byte is sent from the interface to the memory:

1. The interface sends the DMA controller a request for DMA service.(MEMW' or MEMR')
2. A Bus request is made to the HOLD pin (active High) on the 8085 microprocessor and the controller gains control of the bus.
3. A Bus grant is returned to the DMA controller from the Hold Acknowledge (HLDA) pin (active High) on the 8086 microprocessor.
4. The DMA controller places contents of the address register onto the address bus. **Address register –** It contains the address to specify the desired location in memory.
5. The controller sends the interface a DMA acknowledgment, which tells the interface to put data on the data bus. (For an output it signals the interface to latch the next data placed on the bus.)
6. The data byte is transferred to the memory location indicated by the address bus. (if operation in memory write - MEMW')
7. The interface latches the data. (if operation is memory read- MEMR')

Basic DMA Operation

8. The Bus request is dropped, the HOLD pin goes Low, and the controller relinquishes the bus.
9. The Bus grant from the 8086 microprocessor is dropped and the HLDA pin goes Low.
10. The address register is incremented by 1.
11. The byte count is decremented by 1.
12. If the byte count is non-zero, return to step 1, otherwise stop.

HOLD and HLDA Timing during DMA



- HOLD is sampled in any clocking cycle.
- when the processor recognizes the hold, it stops executing software and enters hold cycles
- HOLD input has higher priority than INTR
- The only microprocessor pin that has a higher priority than a HOLD is the RESET pin
- HLDA becomes active to indicate the processor has placed its buses at high-impedance state.
  - ✓ As seen in timing diagram there are a few clock cycles between the time that HOLD changes and until HLDA changes.
- HLDA output is a signal to the requesting device that the processor has relinquished control of its memory and I/O space.
  - ✓ One could call HOLD input a DMA request input and HLDA output a DMA grant signal.

DMA Data Transfer Options

- **Cycle Steal**
  - ✓ In this mode, DMA gives control of buses to CPU after transfer of every byte.
  - ✓ The DMAC effectively steals cycles from the processor in order to transfer the byte, so single byte transfer is cycle stealing.
  - ✓ Either the DMA controller can use the data bus when the CPU does not need it, or it may force the CPU to temporarily suspend operation. The latter technique is called cycle stealing.
- **Burst Transfer:**
  - ✓ To achieve block transfers, some DMAC's incorporate an automatic sequencing of the value presented on the address bus.
  - ✓ A register is used as a byte count, being decremented for each byte transfer, and upon the byte count reaching zero, the DMAC will release the bus.
  - ✓ When the DMAC operates in burst mode, the CPU is halted for the duration of the data transfer.

DMA Data Transfer Options

- **Hidden DMA:**
- Here, DMA transfers data only when CPU is executing the instruction which does not require the use of buses.
  - ✓ It is possible to perform hidden DMA, which is transparent to the normal operation of the CPU.
  - ✓ In other words, the bus is grabbed by the DMAC when the processor is not using it.
  - ✓ The DMAC monitors the execution of the processor, and when it recognizes the processor executing an instruction which has sufficient empty clock cycles to perform a byte transfer; it waits till the processor is decoding the op code, and then grabs the bus during this time.
  - ✓ The processor is not slowed down, but continues processing normally. Naturally, the data transfer by the DMAC must be completed before the processor starts

Advantages of DMA

- Computer system performance is improved by direct transfer of data between memory and I/O devices, bypassing the CPU.
- CPU is free to perform operations that do not use system buses.
- Quick data transfer because a dedicated piece of hardware transfers data from one computer location to another and only one or two bus read/write cycles are required per piece of data transferred.
- Minimizes latency in servicing a data acquisition device because the dedicated hardware responds more quickly than interrupts and transfer time is short.
- Processor is not used for holding the data transfer activity and is available for other processing activity.
- Also in systems where the processor primarily operates out of its cache, data transfer actually occurring in parallel, thus increasing overall system utilization.

Disadvantages of DMA

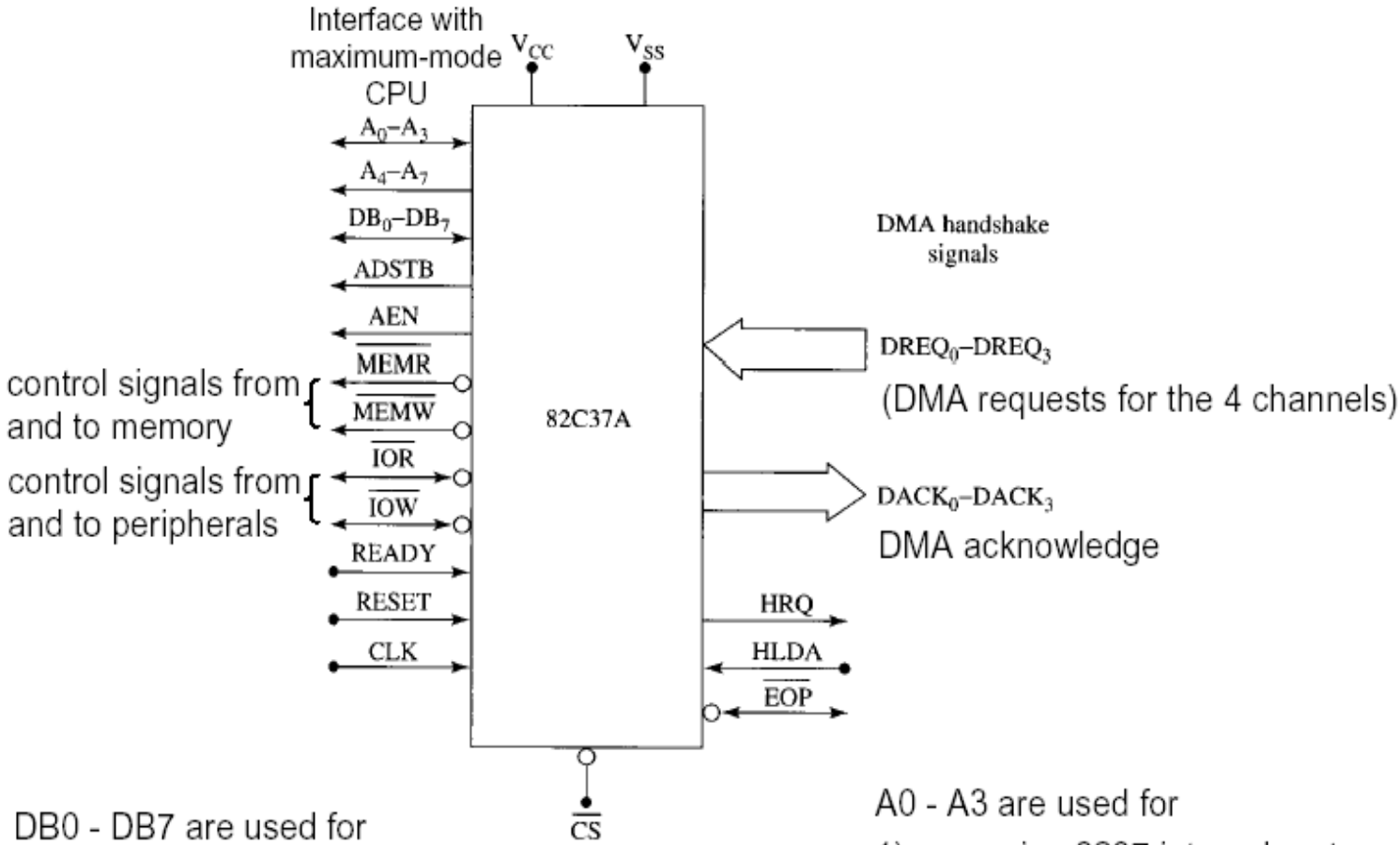
- In case of Burst Mode data transfer, the CPU is rendered inactive for relatively long periods of time.

Programmable DMA Controller - Intel 8237

- It is a device to transfer the data directly between IO device and memory without through the CPU. So it performs a high-speed data transfer between memory and I/O device.
- The features of 8237 is,
  - ✓ The 8237 has four channels and so it can be used to provide DMA to four I/O devices.
  - ✓ Each channel can be independently programmable to transfer up to 64kb of data by DMA.
  - ✓ Each channel can be independently perform read transfer, write transfer.

Programmable DMA Controller - Intel 8237

8237 DMA controller



DB0 - DB7 are used for

1) transfer of data
2) 8237 programming

A0 - A3 are used for

1) accessing 8237 internal ports
2) carrying memory address in DMA read and write operations

8237 Pin Definitions

- **CLK**
  - ✓ **Clock** input is connected to the system clock signal as long as that signal is 5 MHz or less.
  - ✓ In the 8086/8088 system, the clock must be inverted for the proper operation of the 8237
- **CS**
  - ✓ **Chip select** enables 8237 for programming.
  - ✓ The CS pin is normally connected to the output of a decoder.
  - ✓ The decoder does not use the 8086/8088 control signal IO/M(M/IO) because it contains the new memory and I/O control signals (MEMR, MEMW, IOR and IOW).
- **RESET**
  - ✓ The **reset** pin clears the command, status, request, and temporary registers.
  - ✓ It also clears the first/last flip-flop and sets the mask register. This input primes the 8237 so it is disabled until programmed otherwise.

8237 Pin Definitions

- **READY**
  - ✓ Logic 0 on the **ready** input causes the 8237 to enter wait states for slower memory components.
- **HLDA**
  - ✓ A **hold acknowledges** signals 8237 that the microprocessor has relinquished control of the address, data, and control buses.
- **$DREQ_0$–$DREQ_3$**
  - ✓ **DMA request inputs** are used to request a transfer for each of the four DMA channels.
  - ✓ The polarity of these inputs is programmable, so they are either active-high or active-low inputs
- **DB0 ±DB7**
  - ✓ DATA BUS: The Data Bus lines are bidirectional three-state signals connected to the system data bus.
  - ✓ The outputs are enabled in the Program condition during the I/O Read to output the contents of an Address register, a Status register, the Temporary register or a Word Count register to the CPU.

8237 Pin Definitions

- **IOR**
  - ✓ I/O READ: I/O Read is a bidirectional active low three-state line.
  - ✓ In the Idle cycle, it is an input control signal used by the CPU to read the control registers.
  - ✓ In the Active cycle, it is an output control signal used by the 8237A to access data from a peripheral during a DMA Write transfer.
- **IOW**
  - ✓ I/O WRITE: I/O Write is a bidirectional active low three-state line.
  - ✓ In the Idle cycle, it is an input control signal used by the CPU to load information into the 8237A.
  - ✓ In the Active cycle, it is an output control signal used by the 8237A to load data to the peripheral during a DMA Read transfer.
- **A0 ±A3**
  - ✓ ADDRESS: The four least significant address lines are bidirectional three-state signals.
  - ✓ In the Idle cycle they are inputs and are used by the CPU to address the register to be loaded or read.
  - ✓ In the Active cycle they are outputs and provide the lower 4 bits of the output address.

8237 Pin Definitions

- **A4 ±A7 Output**
  - ✓ ADDRESS: The four most significant address lines are three-state outputs and provide 4 bits of address.
  - ✓ These lines are enabled only during the DMA service.
- **HRQ Output**
  - ✓ HOLD REQUEST: This is the Hold Request to the CPU and is used to request control of the system bus.
  - ✓ If the corresponding mask bit is clear, the presence of any valid DREQ causes 8237A to issue the HRQ.
- **DACK0 ±DACK3 Output**
  - ✓ DMA ACKNOWLEDGE: DMA Acknowledge is used to notify the individual peripherals when one has been granted a DMA cycle.
  - ✓ The sense of these lines is programmable. Reset initializes them to active low.
- **AEN**
  - ✓ ADDRESS ENABLE: Address Enable enables the 8-bit latch containing the upper 8 address bits onto the system address bus.
  - ✓ AEN can also be used to disable other system bus drivers during DMA transfers. AEN is active HIGH.

8237 Pin Definitions

- **ADSTB**
  - ✓ ADDRESS STROBE: The active high, Address Strobe is used to strobe the upper address byte into an external latch.
- **MEMR**
  - ✓ MEMORY READ: The Memory Read signal is an active low three-state output used to access data from the selected memory location during a DMA Read or a memory-to-memory transfer.
- **MEMW**
  - ✓ MEMORY WRITE: The Memory Write is an active low three-state output used to write data to the selected memory location during a DMA Write or a memory-to-memory transfer.
- **EOP**
  - ✓ End of Process: At the end of the data transfer the DMA asserts **EOP**.

DMA Interfacing with Microprocessor

- In I/O data transfer data is transferred by using microprocessor. The microprocessor will read data from I/O device and then will write data to memory
- In this case there are two operations for single data transfer.
  - ✓ If the data is less, then microprocessor will not waste its time; transferring data from I/O to memory or back.
  - ✓ But suppose, data is huge, then the transfer rate from I/O to memory or back will slow down because of microprocessor intervention.
    - ❖ In such case, to speed up the process of transferring the data, we can think, Can I/O have direct access to memory and the answer is, yes.
- It can have Direct memory access (DMA), but under Supervision.
  - ✓ The device which supervises, data transfer is named as DMA controller.
- Now let's have diagrammatic representation of the scheme, which depicts microprocessor, DMA controller, memory and I/O device.

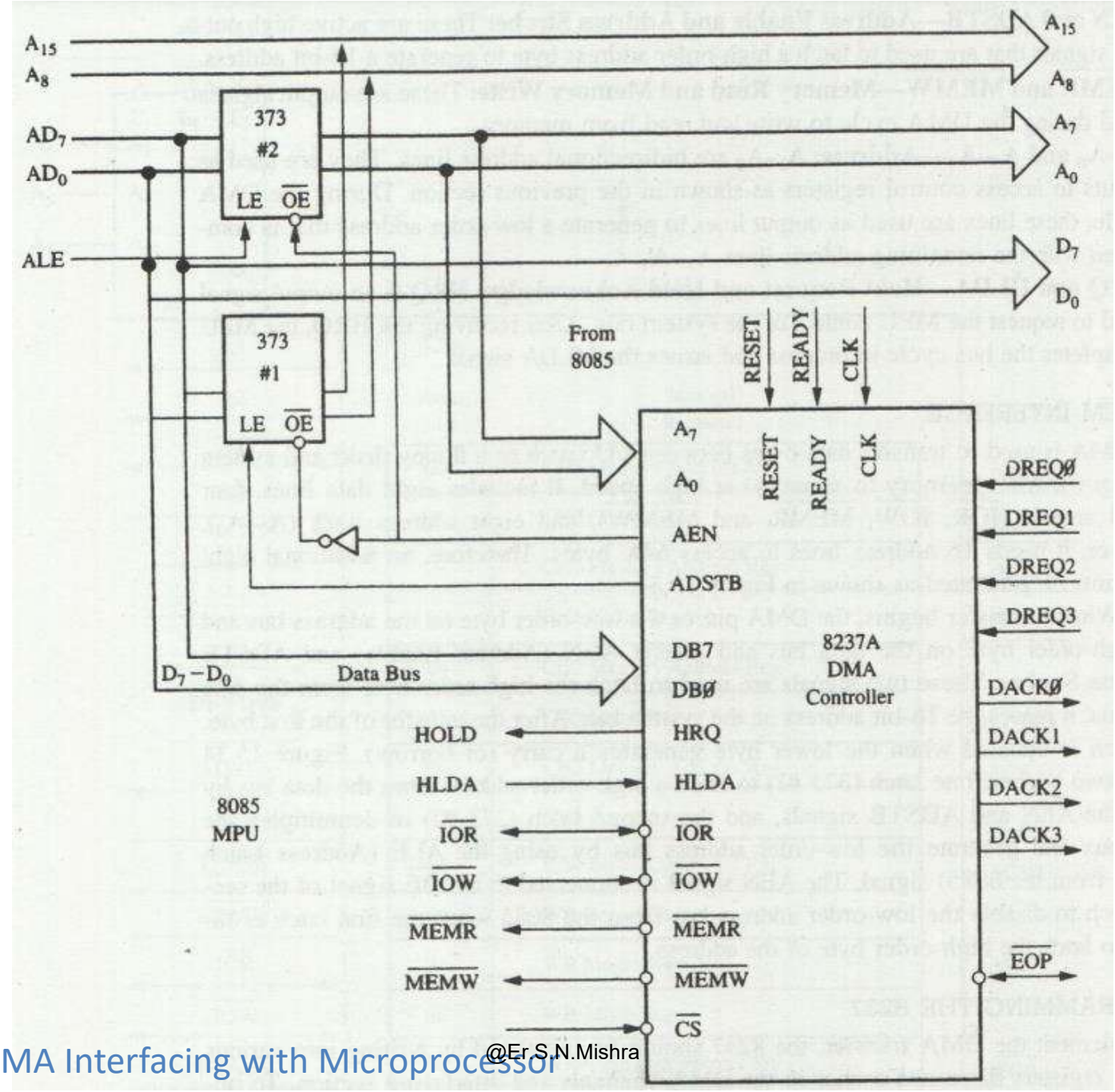


Fig: DMA Interfacing with Microprocessor

DMA Interfacing with Microprocessor

- The DMA is used to transfer data bytes between I/O (such as floppy disk) and system memory (or from memory to memory) at high speed.
- It includes eight data lines, four control signals (IOR, IOW, MEMR, and MEMW), and eight address lines (A7-A0).
  - ✓ However, it needs 16 address lines to access 64K bytes.
  - ✓ Therefore, an additional eight lines must be generated as shown in figure
- When a transfer begins, the DMA places the low-order byte on the address bus and the high-order byte on the data bus and asserts AEN (Address Enable) and ADSTB (Address Strobe).
  - ✓ These two signals are used to latch the high-order byte from the data bus: thus, it places the 16-bit address on the system bus.
- After the transfer of first byte, the latch is updated when the lower byte generates a carry (or borrows).
- Figure shows two latches:
  - ✓ One latch (373 #1) to latch a high-order address from the data bus by using the AEN and ADSTB signals, and
  - ✓ The second latch (373 #2) to demultiplex the 8085 bus and generate the low-order address bus by using the ALE (Address Latch Enable from the 8085) signal.
- The AEN signal is connected to the OE signal of the second latch to disable the low-order address bus from the 8085 when the first latch is enabled to latch the high-order byte of address.

Interrupts

- Interrupt is signals send by an external device to the processor, to request the processor to perform a particular task or work.
- Mainly in the microprocessor based system the interrupts are used for data transfer between the peripheral and the microprocessor.
- The processor will check the interrupts always at the 2nd T-state of last machine cycle.
- If there is any interrupt it accept the interrupt and send the INTA (active low) signal to the peripheral.
- The vectored address of particular interrupt is stored in program counter.
- The processor executes an interrupt service routine (ISR) addressed in program counter.
- It returned to main program by RET instruction.

Interrupt Operations

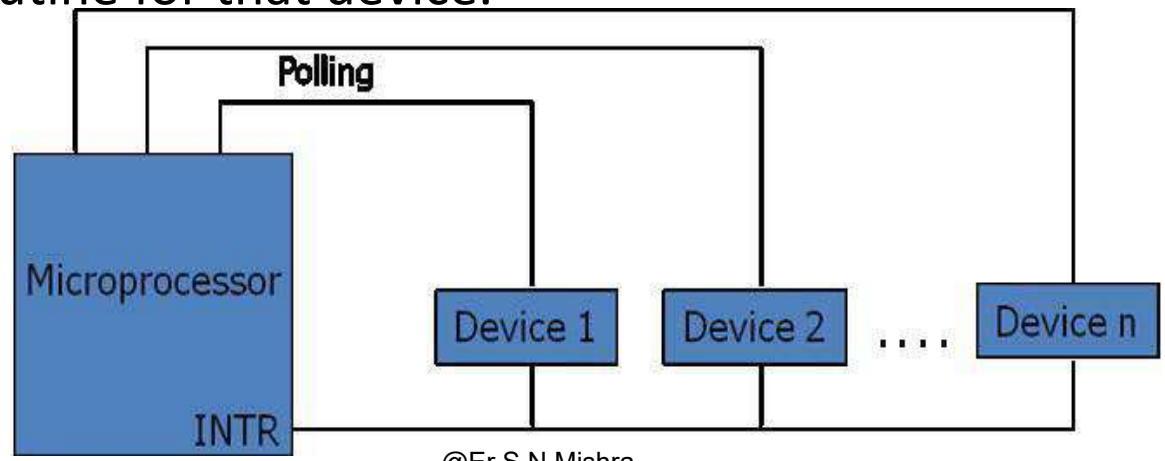
- The transfer of data between the microprocessor and input /output devices takes place using various modes of operations like programmed I/O, interrupt I/O and direct memory access.
- In programmed I/O, the processor has to wait for a long time until I/O module is ready for operation.
  - ✓ So the performance of entire system degraded.
  - ✓ To remove this problem CPU can issue an I/O command to the I/O module and then go to do some useful works.
- The I/O device will then interrupt the CPU to request service when it is ready to exchange data with CPU.
  - ✓ In response to an interrupt, the microprocessor stops executing its current program and calls a procedure which services the interrupt.
- The interrupt is a process of data transfer whereby an external device or a peripheral can inform the processor that it is ready for communication and it requests attention. The response to an interrupt request is directed or controlled by the microprocessor.

Interrupt structures:

- A processor is usually provided with one or more interrupt pins on the chip.
- Therefore a special mechanism is necessary to handle interrupts from several devices that share one of these interrupt lines.
- There are mainly two ways of servicing multiple interrupts which are polled interrupts and daisy chain (vectored) interrupts.

1. Polled interrupts:

- Polled interrupts are handled by using software which is slower than hardware interrupts.
- Here the processor has the general (common) interrupt service routine (ISR) for all devices.
- The priority of the devices is determined by the order in which the routine polls each device.
- The processor checks the starting with the highest priority device.
- Once it determines the source of the interrupt, it branches to the service routine for that device.



1. Polled interrupts:

- Here several external devices are connected to a single interrupt line (INTR) of the microprocessor.
- When INTR signal goes up, the processor saves the contents of PC and other registers and then branches to an address defined by the manufactures of the processor.
- The user can write a program at this address to find the source of the interrupt by starting the polled from highest priority device.

2. Daisy chain (vectored) interrupt:

- This is hardware concept of handling the multiple interrupts. In this technique, the devices are connected in a chain fashion as shown in figure below for setting up the priority system.

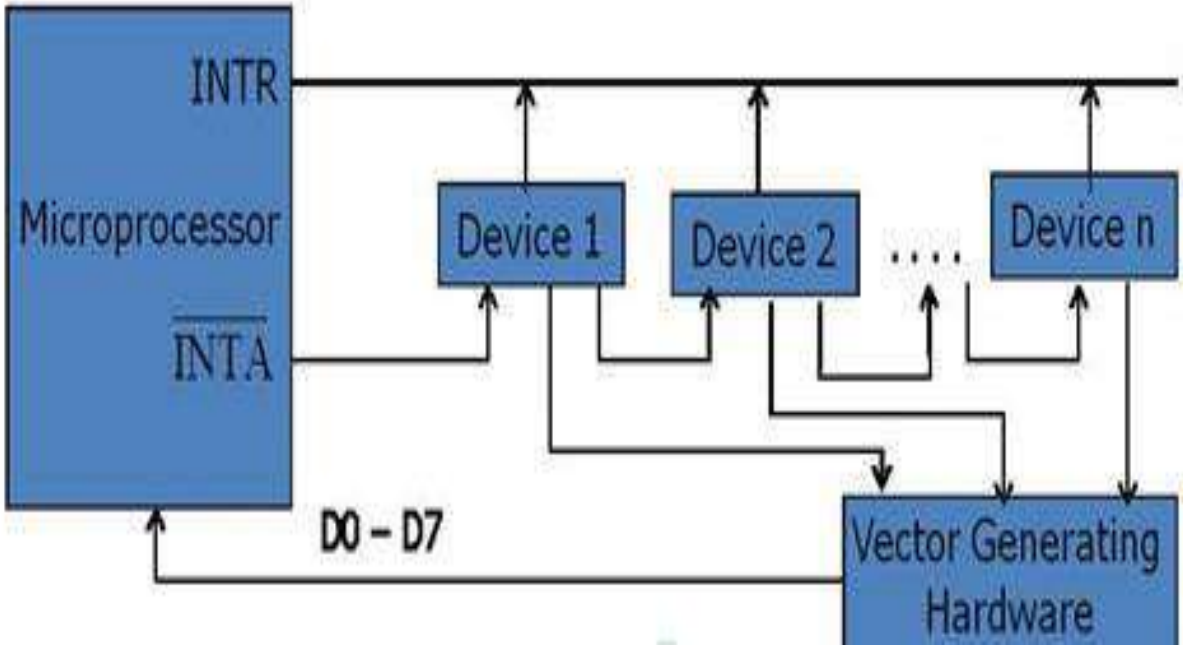


Fig: Vectored (Daisy Chain) Interrupt

2. Daisy chain (vectored) interrupt:

- Here the device with the highest priority placed in the first position, followed by lower priority devices.
- Suppose that one or more devices interrupt the processor at a time.
- In response, the processor saves its current status and then generates an interrupt acknowledge (INTA) signal to the highest priority device, which is device 1 in our case.
- If this device has generated the interrupt it will accept the INTA signal from the processor; otherwise, it will pass INTA on to the next device until the INTA is accepted by the interrupting device.
- Once accepted, the device provides a means to the processor for finding the interrupt address vector using external hardware.
- Usually the requesting device responds by placing a word on the data lines.
- With the help of hardware it generates interrupts vector address. This word is referred to as vector, which the processor used as a pointer to the appropriate device service routine.
- This avoids the need to execute a general interrupt service routine first. So this technique is also referred to as vectored interrupts.

@Dr.S.N.Mishra

Basic Interrupt Processing

- The occurrence of interrupt triggers a number of events, both in processor hardware and in software.
- The interrupt driven I/O operation takes the following steps.
- The I/O unit issues an interrupt signal to the processor for exchange of data between them.
- The processor finishes execution of the current instruction before responding to the interrupt.
- The processor sends an acknowledgement signal to the device that it issued the interrupt.
- The processor transfers its control to the requested routine called Interrupt Service Routine (ISR) by saving the contents of program status word (PSW) and program counter (PC).
- The processor now loads the PC with the location of interrupt service routine and the fetches the instructions. The result is transferred to the interrupt handler program.
- When interrupt processing is completed, the saved register‘s value are retrieved from the stack and restored to the register.
- Finally it restores the PSW and PC values from the stack.

External interrupts:

- These interrupts are initiated by external devices such as A/D converters and classified on following types.
- **Maskable interrupt :**
- It can be enabled or disabled by executing instructions such as EI and DI. In 8085, EI sets the interrupt enable flip flop and enables the interrupt process. DI resets the interrupt enable flip flop and
  - disables the interrupt.
- **Non-maskable interrupt:**
  - It has higher priority over maskable interrupt and cannot be enabled or disabled by the instructions.

Internal Interrupts:

- These are indicated internally by exceptional conditions such as overflow, divide by zero, and execution of illegal op-code.
- The user usually writes a service routine to take correction measures and to provide an indication in order to inform the user that exceptional condition has occurred.
- There can also be activated by execution of TRAP instruction.
  - ✓ This interrupt means TRAP is useful for operating the microprocessor in single step mode and hence important in debugging.
- These interrupts are used by using software to call the function of an operating system.
- Software interrupts are shorter than subroutine calls and they do not need the calling program to know the operating system's address in memory.

Interrupt Service Routine (ISR)

- An interrupt service routine (ISR) is a software routine that hardware invokes in response to an interrupt.
- ISRs examine an interrupt and determine how to handle it.
- ISRs handle the interrupt, and then return a logical interrupt value.
- Its central purpose is to process the interrupt and then return control to the main program.
- An ISR must perform very fast to avoid slowing down the operation of the device and the operation of all lower priority ISRs.
- As in procedures, the last instruction in an ISR should be iret.

Interrupts in 8085 microprocessor

- When microprocessor receives any interrupt signal from peripheral(s) which are requesting its services, it stops its current execution and program control is transferred to a sub-routine by generating CALL signal and after executing sub-routine by generating RET signal again program control is transferred to main program from where it had stopped.
- When microprocessor receives interrupt signals, it sends an acknowledgement (INTA') to the peripheral which is requesting for its service.
- Interrupts in 8085 processor can be classified into various categories based on different parameters:
  - **Hardware and Software Interrupts**
  - **Vectored and Non Vectored Interrupt**
  - **Maskable and Non Maskable Interrupt**

Hardware and Software Interrupts

- When microprocessors receive interrupt signals through pins (hardware) of microprocessor, they are known as ***Hardware Interrupts*.**
- There are 5 Hardware Interrupts in 8085 microprocessor.
  - ✓ They are : INTR, RST 7.5, RST 6.5, RST 5.5 and TRAP.
- ***Software Interrupts*** are those which are inserted in between the program which means these are mnemonics of microprocessor.
- There are 8 software interrupts in 8085 microprocessor.
  - ✓ They are : RST 0, RST 1, RST 2, RST 3, RST 4, RST 5, RST 6 and RST 7

Vectored and Non-Vectored Interrupts

- *Vectored Interrupts* are those which have fixed vector address (starting address of sub-routine) and after executing these, program control is transferred to that address.
- Vector Addresses are calculated by the formula 8 * TYPE

INTERRUPT	VECTOR ADDRESS
TRAP (RST 4.5)	24 H
RST 5.5	2C H
RST 6.5	34 H
RST 7.5	3C H

Vectored and Non-Vectored Interrupts

- For Software interrupts vector addresses are given by:

INTERRUPT	VECTOR ADDRESS
RST 0	00 H
RST 1	08 H
RST 2	10 H
RST 3	18 H
RST 4	20 H
RST 5	28 H
RST 6	30 H
RST 7	38 H

Vectored and Non-Vectored Interrupts

- *Non-Vectored Interrupts* are those in which vector address is not predefined.
- The interrupting device gives the address of sub-routine for these interrupts.
- *INTR* is the only non-vectored interrupt in 8085 microprocessor.

Maskable and Non-Maskable Interrupts

- ***Maskable Interrupts*** are those which can be disabled or ignored by the microprocessor.
  - ✓ These interrupts are either edge-triggered or level-triggered, so they can be disabled.
  - ✓ *INTR, RST 7.5, RST 6.5, RST 5.5* are Maskable interrupts in 8085 microprocessor.
- ***Non-Maskable Interrupts*** are those which cannot be disabled or ignored by microprocessor.
  - ✓ *TRAP* is a Non-Maskable interrupt.
  - ✓ It consists of both level as well as edge triggering and is used in critical power failure conditions.

Priority of Interrupts

- When microprocessor receives multiple interrupt requests simultaneously, it will execute the interrupt service request (ISR) according to the priority of the interrupts.

Instruction for Interrupts

- Enable Interrupt (EI) :
  - ✓ The interrupt enable flip-flop is set and all interrupts are enabled following the execution of next instruction followed by EI.
  - ✓ No flags are affected.
  - ✓ After a system reset, the interrupt enable flip-flop is reset, thus disabling the interrupts.
  - ✓ This instruction is necessary to enable the interrupts again (except TRAP).
- Disable Interrupt (DI):
  - ✓ This instruction is used to reset the value of enable flip-flop hence disabling all the interrupts.
  - ✓ No flags are affected by this instruction.

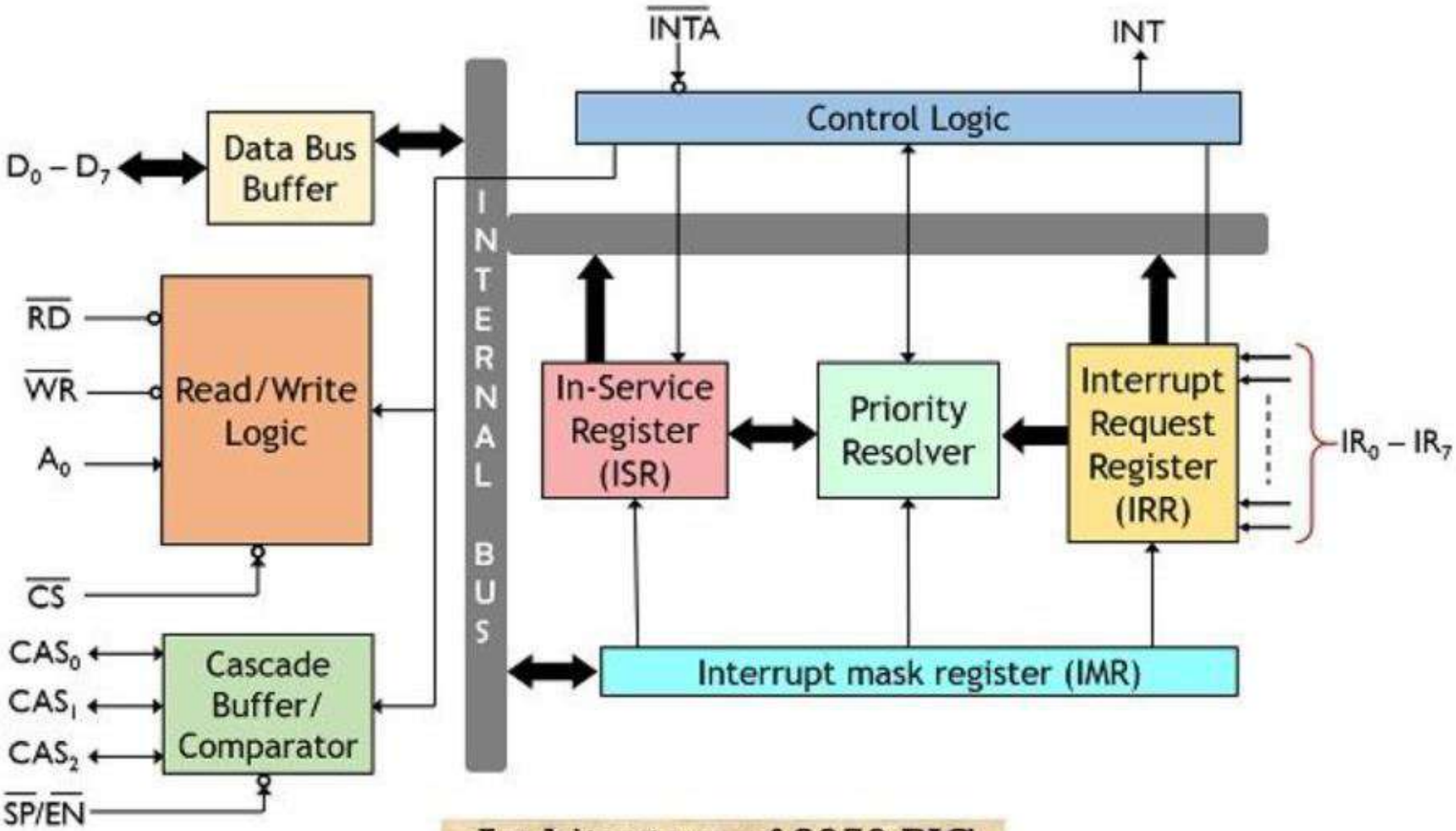
The 8259A Programmable Interrupt Controller

- The 8259A programmable interrupt controller designed to work with Intel microprocessors 8085, 8086 and 8088.
- The 8259A interrupt controller can:
  - ❖ Manage eight interrupts according to the instructions written into its control registers. This is equivalent to proving eight interrupt pins on the processor in place of one INTR (8085) pin.
  - ❖ Vector can interrupt request anywhere in the memory map. However, all eight interrupts are spaced at the interval of either four or eight locations. This eliminates all the major drawback of the 8085 interrupts in which all interrupts are vectored to memory locations on page 00H
  - ❖ Resolve eight levels of interrupt priorities in a variety of modes, such as fully nested mode, automatic rotation mode, and specific rotation mode.
  - ❖ Mask each interrupt request individually.
  - ❖ Read the status of pending interrupts, in-service interrupts, and masked interrupts.
  - ❖ Be set up to accept either the level-triggered or the edge-triggered interrupt request
  - ❖ Be expanded to 64 priority levels by cascading additional 8259As.
  - ❖ Be set up to work with either the 8085 microprocessor mode or the 8086/8088 microprocessor mode

The 8259A Programmable Interrupt Controller

- Figure shows the internal block diagram of the 8259A.
- It includes eight blocks: control logic, Read/Write logic, data bus buffer, three registers (IRR, ISR and IMR), priority resolver, and cascade buffer.
- This diagram shows all the elements of a programmable device, plus additional blocks.
- The functions of these blocks are given below:

Block Diagram of The 8259A



Architecture of 8259 PIC

Block Diagram of The 8259A

- **DATA BUS BUFFER**

✓ 8259 has tri-stated bidirectional 8-bit data bus buffer (i.e., D_0 to D_7) that interfaces with the internal bus of the processor.

✓ The 8085 microprocessor sends/ receives, control or status words to/ from the 8259 using data bus buffer..

- **READ/WRITE LOGIC**

✓ This unit is responsible for controlling the internal read-write operations of the system.

✓ It holds initialization command word register and operation command word register inside which various control formats exist that are needed for the device operation.

✓ When the address line A0 is at logic 0,the controller is selected to write a command or read a status.

✓ A low signal at CS i.e., chip select shows that now the communication has been set up between the processor and 8259.

✓ The chip select logic and A0 determine the port address of the controller.

Block Diagram of The 8259A

- **CONTROL LOGIC**

✓ This unit is the heart of the architecture of 8259.

✓ It controls the overall operation of the system by sending the INTR signal to the processor whenever an interrupt request is generated.

- Also, it receives INTA signal by the processor when microprocessor demands for the address of the interrupt service routine.
- The control logic is responsible for sending the address of the desired interrupt service routine through the data bus.

- **Interrupt request register (IRR)**:

✓ This unit stores the interrupt requests generated by the peripheral devices.

✓ We know that 8259 has 8 interrupt request pins (i.e., IR_0 to IR_7).

✓ So, the unit can store 8 interrupt requests that are requesting the service from the processor.

Block Diagram of The 8259A

- **Priority Resolver**:

✓ This logic unit decides that among all the interrupt request present in the IRR which holds the highest priority and needs to be executed first.

✓ Suppose at the time of servicing an interrupt, another incoming interrupt request gets generated then that request will be ignored as the one in-service is holding the highest priority.

✓ But in case the incoming request has greater priority than the one which is being in current execution then that respective bit will be set in ISR and INTR signal is sent to the microprocessor.

✓ This simply means that only the interrupt holding the highest priority will be forwarded by the 8259 to the processor.

Block Diagram of The 8259A

- **In-service register:**

✓ Here the name of the unit is itself indicating the operation performed by it.

✓ This register unit stores the interrupts which are currently being executed by the processor.

✓ The priority resolver sets each bit of ISR and after getting interrupt word command by the processor, the bits get reset.

✓ As the processor holds the ability to directly read the status of in-service register.

- **Interrupt mask register**:

✓ This register unit holds the masking bit of those interrupts which are to be masked.

✓ Through operation command word (OCW) the processor sends the required information and programs the interrupt mask register.

Block Diagram of The 8259A :Cascade Buffer/Comparator

- As we have already discussed that by cascading multiple 8259, the number of interrupts handled by 8259 can be expanded up to 64.
- The unit allows the comparison of IDs of different 8259s cascaded together.
- It permits the operation of the system in two modes: **master mode** and **slave mode**.
- In the master mode of operation, it acts as a cascaded buffer Whereas in slave mode, this unit acts as a comparator.
- Among the various cascaded 8259, one 8259 directly handles the interrupts by forming a connection with the processor and it is known to be master 8259. While the other 8259s that interrupts the master 8259 are known as slave 8259.
- Each of the 8259s can be separately programmed as all of them holds a specific address.
- The cascading pins of the master 8259, CAS0, CAS1 and CAS2 forms connection with the corresponding pins of slave 8259s.
- For the slave devices, these pins act as input pins while for a master device these acts as output pins.
- An active-low signal at SP/EN for a device shows that it is operating in slave mode.

8259A : Interrupt Operation

- To implement interrupts, the Interrupt Enable flip-flop in the microprocessor should be enabled by writing the EI instruction, and the 8259A should be initialized by writing control words in the control register.
- The 8259A requires two types of control words: Initialization Command Words (ICWs) and Operational Command Words (OCWs).
- The ICWs are used to set up the proper conditions and specify RST vector addresses.
- The OCWs are used to perform functions such as masking interrupts, setting up status-read operations, etc.
- After the 8259A is initialized, the following sequence of events occurs when one or more interrupt request lines go high.
  - The IRR stores the requests.
  - The priority resolver checks three registers: the IRR for interrupt requests, the IMR for masking bits, and the ISR for the interrupt request being served. It resolves the priority and sets the INT high when appropriate.
  - The MPU acknowledges the interrupt by sending INTA.

8259A : Interrupt Operation

- After the INTA is received, the appropriate priority bit in the ISR is set to indicate which interrupt level is being served, and the corresponding bit in the IRR is reset to indicate that the request is accepted. Then, the opcode for the CALL instruction is placed on the data bus.
- When the MPU decodes the CALL instruction, it places two or more INTA signals on the data bus.
- When 8259A receives the second INTA, it places the low-order byte of the CALL address on the data bus. At the third INTA, it places the high-order byte on the data bus. The CALL address is the vector memory location for the interrupt: this address is placed in the control register during the initialization.
- During the third INTA pulse, the ISR bit is reset either automatically (Automatic-End-of-Interrupt-AEOI) or by a command word that must be issued at the end.

Priority Modes and Other Features

- Many types of priority modes are available under software control in the 8259A, and they can be changed dynamically during the program by writing appropriate command words.
- Commonly used priority modes are discussed below:

❖ **Fully Nested Mode:**

- ✓ This is a general-purpose mode in which all IRS (interrupt Requests) are arranged from highest to lowest, with IR0 as the highest and IR7 as the lowest.
- ✓ In addition, any IR can be assigned the highest priority in this mode; the priority sequence will then begin at that IR. In the example below, IR4 has the highest priority, and IR3 has the lowest priority:

IR0	IR1	IR2	IR3	IR4	IR2	IR3	IR4
4	5	6	7	0	1	2	3

❖ **Automatic Rotation Mode:**

- ✓ In this mode, a device, after being serviced, receives the lowest priority.
- ✓ Assuming that the IR2 has just been serviced, it will receive the seventh priority, as shown below:

IR0	IR1	IR2	IR3	IR4	IR2	IR3	IR4
1	6	7	0	1	2	3	4

❖ **Specific Rotation Mode:** This mode is similar to the automatic rotation mode, except that the user can select any IR for the lowest priority